

Отчёт по выполнению лабораторной работы №10

Работа с ядром Linux.

Коровкин Н. М.

30 октября 2025

Российский университет дружбы народов, Москва, Россия

Информация

- Коровкин Никита Михайлович
- Студент
- Российский университет дружбы народов
- 1132246835@pfur.ru

Получить навыки работы с утилитами управления модулями ядра операционной системы

1. Продемонстрируйте навыки работы по управлению модулями ядра (см. раздел 10.4.1).
2. Продемонстрируйте навыки работы по загрузке модулей ядра с параметрами

Выполнение лабораторной работы

Я запустил терминал и получил полномочия администратора, введя команду `su -`. После этого я посмотрел, какие устройства подключены к системе и какие модули ядра с ними связаны, используя команду `lspci -k`. (рис. (fig:001?))

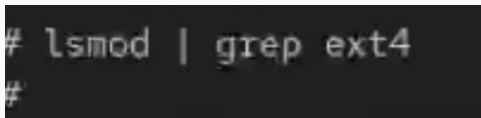
```
Definition Audio Controller (rev 01)
  Subsystem: Red Hat, Inc. QEMU Virtual Machine
  Kernel driver in use: snd_hda_intel
  Kernel modules: snd_hda_intel
00:04.0 USB controller: NEC Corporation uPD720200 USB 3.0 Host Controller (rev 03)
  Subsystem: Red Hat, Inc. QEMU Virtual Machine
  Kernel driver in use: xhci_hcd
00:05.0 USB controller: Red Hat, Inc. QEMU XHCI Host Controller (rev 01)
  Subsystem: Red Hat, Inc. Device 1100
  Kernel driver in use: xhci_hcd
00:06.0 SCSI storage controller: Red Hat, Inc. Virtio block device
  Subsystem: Red Hat, Inc. Device 0002
  Kernel driver in use: virtio-pci
00:07.0 Communication controller: Red Hat, Inc. Virtio console
  Subsystem: Red Hat, Inc. Device 0003
  Kernel driver in use: virtio-pci
```

Выполнение лабораторной работы

В выводе отобразились все устройства — сетевой адаптер, контроллер USB и другие. . Затем я вывел список всех загруженных модулей ядра с помощью команды `lsmod | sort`. Эта команда показала, какие модули сейчас используются системой, а также их взаимозависимости.(рис. (fig:002?))

```
snd_hwdep          20480 1 snd_hda_codec
snd_intel_dspcfg    24576 1 snd_hda_intel
snd_pcm            147456 3 snd_hda_intel,snd_hda_codec,snd_hda_core
snd_seq            110592 7 snd_seq_dummy
snd_seq_device      16384 1 snd_seq
snd_seq_dummy       12288 0
snd_timer           49152 3 snd_seq,snd_hrtimer,snd_pcm
soundcore           16384 1 snd
uinput              24576 1
vfat                 24576 1
virtio_blk           36864 4
virtio_console       40960 2
virtio_dma_buf        12288 1 virtio_gpu
virtio_gpu           81920 2
virtio_mmio          20480 0
```

Чтобы проверить, загружен ли модуль ext4, я ввёл команду `lsmod | grep ext4`. (рис. (fig:003?))

A terminal window with a dark background. The prompt character is '#'. The command 'lsmod | grep ext4' has been entered and is displayed on the line following the prompt. A second prompt character '#' is visible on the line below the command.

```
# lsmod | grep ext4
#
```

Рис. 3: проверка модуля

Для загрузки модуля я использовал команду `modprobe ext4`, после чего снова проверил список модулей.(рис. (fig:004?))

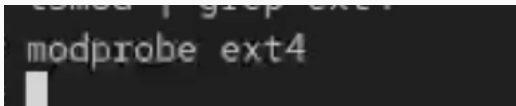
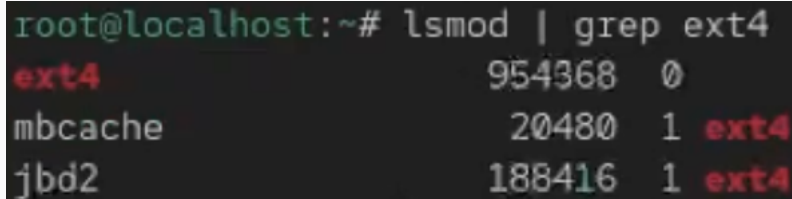


Рис. 4: загрузка модуля

Выполнение лабораторной работы

Теперь ext4 появился в выводе, значит модуль успешно загружен.(рис. (fig:005?))



```
root@localhost:~# lsmod | grep ext4
ext4                954368    0
mbcache             20480     1  ext4
jbd2                188416     1  ext4
```

Рис. 5: модуль загружен

Выполнение лабораторной работы

Далее я посмотрел подробную информацию о модуле ext4 с помощью команды `modinfo ext4`. Перечислю вывод - Rocky kernel signing key - это имя ключа, которым подписан модуль ext4. `sig_key`:

24:2F:36:2D:03:F4:44:18:02:8C:20:2B:26:07:EC:2A:2C:64:31:51

Это отпечаток (fingerprint) открытого ключа, которым модуль был подписан.

Он используется для проверки, что подпись сделана доверенным ключом.

`sig_hashalgo: sha256` — алгоритм хеширования, применяемый при создании подписи. `signature`: используется ядром Linux для проверки, что модуль не был изменён после компиляции и что он подписан доверенным ключом. (рис. (fig:006?))

```
signer:      Rocky kernel signing key
sig_key:     24:2F:36:2D:03:F4:44:18:02:8C:20:2B:26:07:EC:2A:2C:64:31:51
sig_hashalgo: sha256
signature:   0B:DC:0C:26:5E:C6:CD:85:E1:DC:3B:90:85:24:08:37:96:7C:C2:43:
             C4:06:F3:64:6E:1E:2B:FA:6B:37:04:F4:76:80:8E:AD:EB:27:A4:67:
```

Выполнение лабораторной работы

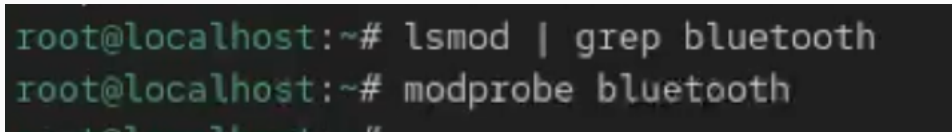
После этого я попробовал выгрузить модуль ext4 командой `modprobe -r ext4`. Иногда ядро не позволяет выгрузить модуль, если он используется системой (например, когда файловая система ext4 используется для текущего корневого раздела). Поэтому команда может не сработать с первого раза. В этом случае система выдаёт сообщение о том, что модуль занят. Я также попробовал выгрузить модуль xfs командой `modprobe -r xfs`. (рис. (fig:007?))

```
root@localhost:~# modprobe -r ext4
root@localhost:~# modprobe -r ext4
root@localhost:~# modprobe -r ext4
root@localhost:~# modprobe -r ext4
root@localhost:~# modprobe -r xfs
modprobe: FATAL: Module xfs is in use.
```

Выполнение лабораторной работы

Затем я снова запустил терминал от имени администратора и проверил, загружен ли модуль Bluetooth `lsmod | grep bluetooth`.

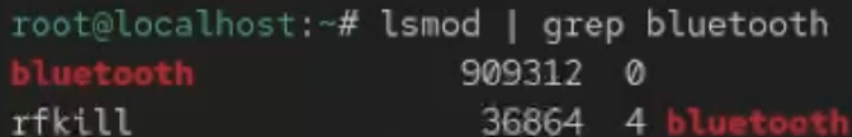
Так как он не был загружен, я добавил его командой `modprobe bluetooth`, после чего убедился, что модуль действительно появился в списке активных.(рис. (fig:008?))



```
root@localhost:~# lsmod | grep bluetooth
root@localhost:~# modprobe bluetooth
```

Рис. 8: модуль загружен

Модуль присутствует. Чтобы узнать, какие ещё модули связаны с Bluetooth, я снова использовал `lsmod | grep bluetooth`. В выводе появилось `rfkill` (рис. (fig:009?))



```
root@localhost:~# lsmod | grep bluetooth
bluetooth          909312  0
rfkill             36864  4 bluetooth
```

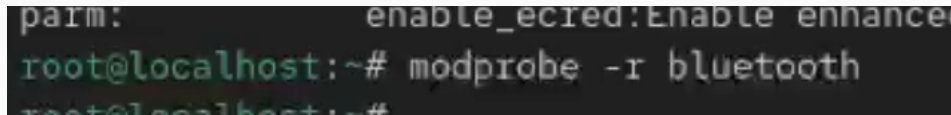
Рис. 9: модуль загружен

Выполнение лабораторной работы

Затем я посмотрел информацию о самом модуле bluetooth с помощью modinfo bluetooth.Здесь также отображаются ключ, которыми подписан этот модуль и присутствуют его параметры.(рис. (fig:010?))

```
signature: 11:56:48:F7:C0:BF:16:5F:84:E9:4D:9E:0B:6F:A6:12:81:5A:6E:57:
            26:44:2C:7B:6F:88:42:2D:9D:9D:BC:D0:E8:95:B0:4D:B6:0E:3F:38:
            38:97:EF:8F:6D:9E:7F:00:F6:B9:E3:3C:CA:E0:39:44:2D:CD:5C:07:
            E0:5A:91:88:5D:78:C9:C3:73:AD:EB:E7:51:CE:AE:F1:FB:F9:B0:98:
            2C:10:B6:99:8E:33:64:4F:22:FA:4C:8B:0B:E8:B7:7C:5D:46:E6:32:
            FC:0A:4A:D0:61:09:7C:1E:59:E8:C2:3F:FC:92:1E:DC:A2:67:C5:CD:
            38:E5:66:FA:6A:59:DF:C9:0E:7F:B9:15:AD:90:6D:33:B1:FC:E9:7C:
            09:7D:CF:38:B4:50:D0:AB:BB:8F:82:FC:01:3D:CF:AC:42:65:1D:07:
            27:84:EE:F0:E9:A0:55:72:3C:0C:04:47:83:31:D4:5A:61:60:AB:5B:
            6B:7B:58:DB:68:C4:40:0E:81:2A:C3:5A:3E:05:84:51:0F:63:89:09:
            0B:21:8C:33:34:C8:23:99:4D:41:D7:D1:E1:F2:F9:10:7F:0C:97:18:
            82:4D:55:67:5F:35:68:DF:E4:F4:20:77:8D:8D:5C:38:01:6D:6D:35:
            6A:96:5A:C0:C0:1B:95:AA:81:72:59:40:28:2D:98:DE:05:D8:32:C4:
            D4:1E:61:00:FF:64:22:67:7F:C2:F9:14:77:F8:42:79:EE:96:BC:1C:
            67:70:C6:11:8E:61:39:22:76:ED:D3:76:3B:8B:75:91:03:9E:55:15:
            77:00:52:C5:7F:13:16:5B:19:90:5A:A9:6E:30:9E:4C:C4:57:E9:EB:
            14:34:82:FB:D1:9D:68:99:83:CF:F5:DF:A7:D4:FC:65:58:3F:BC:F9:
```

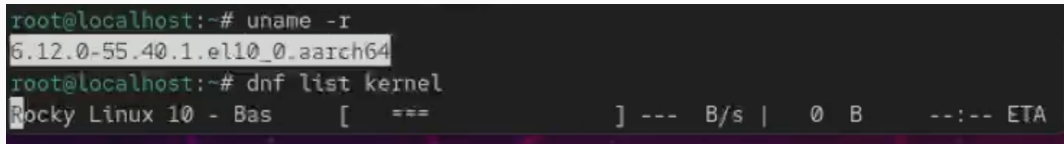
После проверки я выгрузил модуль командой `modprobe -r bluetooth`. (рис. (fig:011?))

A terminal window with a dark background. The prompt is 'root@localhost:~#'. The command 'modprobe -r bluetooth' is entered and executed. The output is partially visible as 'parm: enable_ecred:Enable enhance'.

```
parm: enable_ecred:Enable enhance
root@localhost:~# modprobe -r bluetooth
root@localhost:~#
```

Рис. 11: выгружаю модуль

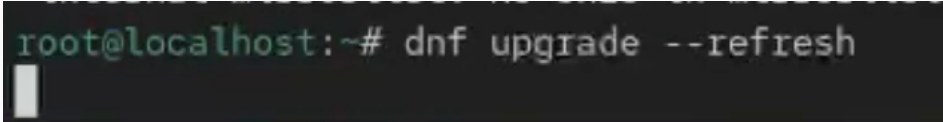
Далее я снова открыл терминал с правами администратора и посмотрел текущую версию ядра с помощью команды `uname -r`. После этого я вывел список всех пакетов, относящихся к ядру, используя `dnf list kernel`. (рис. (fig:012?))

A terminal window with a dark background. The first command is 'root@localhost:~# uname -r' and the output is '6.12.0-55.40.1.el10_0.aarch64'. The second command is 'root@localhost:~# dnf list kernel'. The output shows 'Rocky Linux 10 - Bas' followed by a table header: '[===] --- B/s | 0 B --:-- ETA'.

```
root@localhost:~# uname -r
6.12.0-55.40.1.el10_0.aarch64
root@localhost:~# dnf list kernel
Rocky Linux 10 - Bas      [ === ] --- B/s | 0 B --:-- ETA
```

Рис. 12: список пакетов

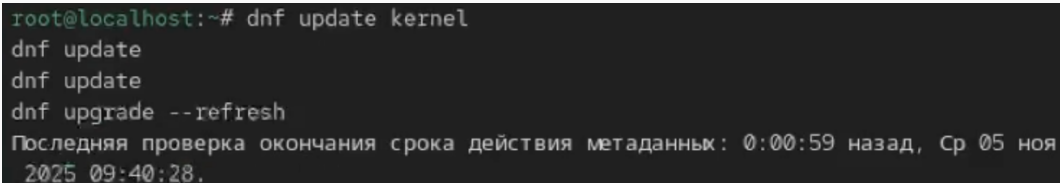
Чтобы убедиться, что система полностью обновлена, я выполнил `dnf upgrade --refresh`. Это важно перед обновлением ядра, чтобы избежать конфликтов.(рис. (fig:013?))

A terminal window with a dark background. The prompt is 'root@localhost:~#'. The command 'dnf upgrade --refresh' is being typed. A white cursor is visible at the end of the command.

```
root@localhost:~# dnf upgrade --refresh
```

Рис. 13: обновление - подготовка

После этого я обновил ядро и саму систему.(рис. (fig:014?))



```
root@localhost:~# dnf update kernel
dnf update
dnf update
dnf upgrade --refresh
Последняя проверка окончания срока действия метаданных: 0:00:59 назад, Ср 05 ноя
2025 09:40:28.
```

Рис. 14: обновление

Выполнение лабораторной работы

Когда обновление завершилось, я перезагрузил компьютер и при загрузке выбрал новое ядро в меню.(рис. (fig:015?))

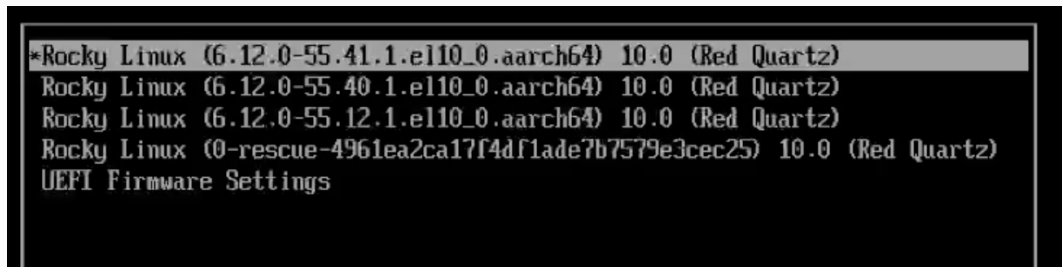


Рис. 15: p

Выполнение лабораторной работы

После входа в систему я снова проверил версию ядра через `uname -r` — она изменилась, значит обновление прошло успешно. Команда `hostnamectl` также показывает текущую версию ядра и другую системную информацию. (рис. (fig:016?))

```
nmkorovkin@localhost:~$ uname -r
hostnamectl
6.12.0-55.41.1.el10_0.aarch64
  Static hostname: (unset)
  Transient hostname: localhost
    Icon name: computer-vm
    Chassis: vm 🖥️
    Machine ID: 4961ea2ca17f4df1ade7b7579e3cec25
    Boot ID: a5403502589e420686ee7bc1d907da9e
  Virtualization: qemu
  Operating System: Rocky Linux 10.0 (Red Quartz)
    CPE OS Name: cpe:/o:rocky:rocky:10::baseos
    OS Support End: Thu 2035-05-31
  OS Support Remaining: 9y 6month 3w 2d
    Kernel: Linux 6.12.0-55.41.1.el10_0.aarch64
    Architecture: arm64
```

Ответ на контрольные вопросы

1. Какая команда показывает текущую версию ядра?☒Команда `uname -r`.
2. Как можно посмотреть более подробную информацию о текущей версии ядра?☒С помощью команды `hostnamectl`, которая показывает не только версию ядра, но и данные о системе, хосте и архитектуре.
3. Какая команда показывает список загруженных модулей ядра?☒Команда `lsmod`.
4. Какая команда позволяет определять параметры модуля ядра?☒Команда `modinfo имя_модуля`.
5. Как выгрузить модуль ядра?☒Командой `modprobe -r имя_модуля`.
6. Что можно сделать, если при попытке выгрузить модуль появляется ошибка?☒Ошибка обычно означает, что модуль используется системой. В этом случае нужно сначала остановить процесс или отключить устройство, которое использует модуль, а затем повторить команду.
7. Как определить, какие параметры модуля ядра

В ходе выполнения данной лабораторной работы были получены навыки управления ядром.

